Process Verification

Lab 6

Quentin Meneghini

11th May 2026

Facultat d'Informàtica de Barcelona

Contents

Chapter 1

Design Overview

1.1 Controller

A dual-master memory controller. Master 0 (CPU) has fixed priority over Master 1 (DMA). Synchronous writes on clock, and combinational reads.

1.1.1 Parameters

There are two parameters to this dut:

- `ADDR_WIDTH` : address size.
- `DATA_WIDTH` : data width.

1.1.2 Functional Description

Condition	Behavior
Only one master valid	That master is serviced immediately
Both masters valid	Master 0 is serviced (priority). Master 1 is not granted
No master valid	Controller idle

Table 1.1: Controller Behavior

- **Reads**: 0 clock cycles (combinational —immediate once address & grant are stable).
- **Writes**: 1 clock cycle (synchronous —update happens on next rising edge when grant & we are true).

1.1.3 Interface

Signal	Direction	Width	Description
`clk`	Input	1	Clock signal
`rst_n`	Input	1	Active-low synchronous reset
`addr0`	Input	ADDR_WIDTH	Master 0 address
`wdata0`	Input	DATA_WIDTH	Master 0 data
`rdata0`	Output	DATA_WIDTH	Master 0 data
`we0`	Input	1	Master 0 write enable
`req0`	Input	1	Master 0 read request
`gnt0`	Output	1	Master 0 grant
`addr1`	Input	ADDR_WIDTH	Master 1 address
`wdata1`	Input	DATA_WIDTH	Master 1 data
`rdata1`	Output	DATA_WIDTH	Master 1 data
`we1`	Input	1	Master 1 write enable
`req1`	Input	1	Master 1 read request
`gnt1`	Output	1	Master 1 grant

Table 1.2: Controller signals

Chapter 2

Plan

The main points that have been followed to reach the final design are:

1. Having too many signals make debugging extremely slow and burdensome, so I would change those into a compact and reusable interface.
2. Introducing a clocking block will surely make the timing management more easy and less prone to errors.
3. The transaction has too many not needed signals and wraps both masters. I will separate the masters and use strictly useful signals, leaving simulation control to UVM.
4. Instead of a long and repetitive generator, I will make simple building blocks that can be reused across tests and make the creation of new test fast and easy.
5. After making the timing management automatic, dividing the agents into two separate ones should be easy and make the structure extremely scalable.
6. I will introduce the concept of coverage and hardware checking.

Chapter 3

Diagram

For simplicity, the main diagram (figure 3.1) has the main structural pieces separated, while they follow a hierarchical structure. That structure is shown in figure 3.2.

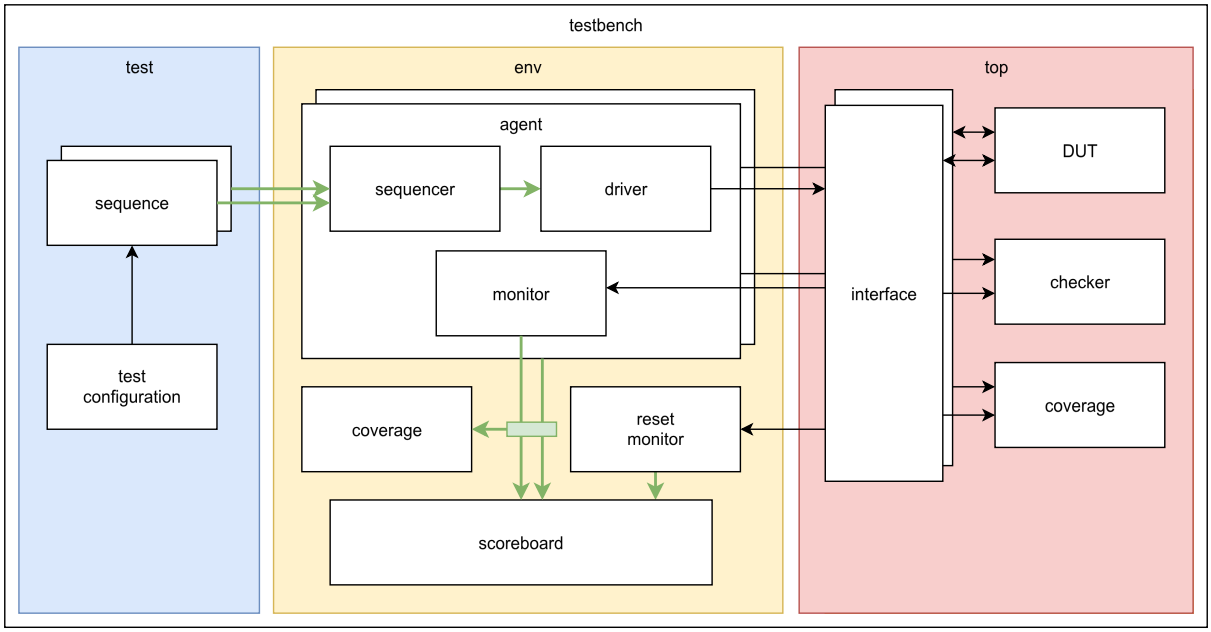

Figure 3.1: Diagram of the structure of the testbench

There are multiple sequences to improve the modularity of the system: random sequence and deterministic write + read.

The green arrows represent the communication that is handled by the UVM ports. The subscriber uses the green rectangle to make the "man in the middle" attack to that communication.

Each interface is connected to its own driver and the pins to a single master in the `DUT`. This improves modularity, but creates a second problem: the concurrence information is lost in this

communication. The scoreboard can not see if 2 consecutive transactions have been executed together.

Both the `checker` and the `coverage` modules are connected to the `DUT` through a binding. This connection means that they are not actually connected to the interface, but incorporated to the module itself, but for simplicity they were connected to the interface here. These two modules are in charge of the concurrent behavior of the two masters connected to the `DUT`.

1. `Testbench`: This component simply wraps around every component and build the final machine.
2. `Top`: This component instantiates the `DUT`, its interfaces the test to be executed and the non UVM modules such as checker and cover.
3. `Test`: This component is responsible for the creation of sequences and the very environment that will perform all UVM transactions.
4. `Environment`: This component is the core of the UVM structure. Here there are the agents (sequencer, driver, and monitor), and the scoreboard.

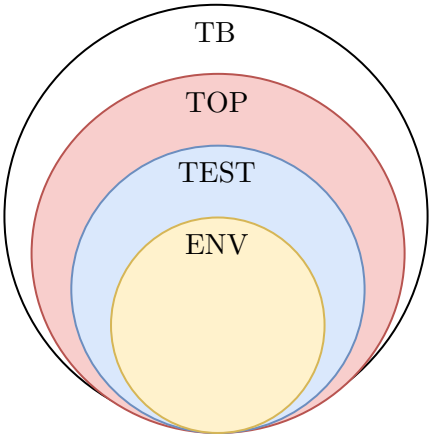

Figure 3.2: Hierarchy of the testbench components

Chapter 4

Result

To covert the classical testbench into a UVM one we modified the general structure of some modules, created new ones, introduced the coverage and added some features to make testing smooth.

4.1 Changes

Here we will introduce all the logic changes from the classical testbench to the UVM one. The changes will be explored in the following subsections.

Transaction (sequence item)

The transaction used in the initial testbench wraps both masters and some unuseful signals such as `use_m*` and `is_end`. These informations will be handled by the UVM structure and the driver idle state. The final product is in charge of only one master and leaves the burden of the two masters conflicts to the sequences. There is a second (outputs) and third (reset) transactions that handle the communication between monitors and scoreboard.

Virtual Interface & Clocking Block

Signals will be abstracted into an `interface` and synchronized with a clocking block (`cb`). Following the transaction division of signals into masters, the `interface` abstract the control of a single master, this will lead to a clearer communication between UVM components. This element is instantiated in the top module, and distributed through the `uvm_config_db`.

```
12 interface ctrl_interface (
13     input logic clk,
14     input logic rst_n
15 );
16
17     logic [ADDR_WIDTH-1:0] addr;
18     logic [DATA_WIDTH-1:0] wdata;
19     logic [DATA_WIDTH-1:0] rdata;
20     logic                  we;
21     logic                  req;
22     logic                  gnt;
23
24     clocking cb @(posedge clk);
25         default input #1step output #0;
26         input rdata, gnt;
27         inout rst_n, addr, wdata, we, req;
28     endclocking
29
30 endinterface
```

Generation (sequence / sequencer)

The generation of transaction was handled with a repetitive function, hard to scale and to verify. Following the UVM architecture this will be handled by sequences that are easy to use, and to change if necessary. Furthermore, the clocking collisions are handled by the UVM sequencer of each agent, so no manual debugging will be necessary. There are two main tests:

- **Deterministic** tests are known before the execution and are used to test a specific feature:
  - **Single master write + read**
  - **Contention tests** where both masters request the memory at the same time
- **Random** tests are used to test the DUT with some general behaviors

Driver

The original module took care of both master at the same time, added clock delays and applied the transactions to the DUT signals. The new driver is compartmentalized to only one master and does not wait useless cycles that would make back to back instruction impossible. It also handles reset sequences to not lose transactions.

The key element of this structure is the `apply` task. This function applies the delay of each transaction, wait for the ack signal, and leaves the enable signal low in case there is no next instruction to be executed. This last feature is a key feature to make the two masters driver work simultaneously without creation of false commands and spin locks.

```
105         task apply(ctrl_input_transaction trans);
106             repeat (trans.delay_cycles) @(posedge ctrl_if.clk);// apply delay
107             ctrl_if.cb.req      <= 1'b1;
108             ctrl_if.cb.we       <= trans.we;
109             ctrl_if.cb.addr     <= trans.addr;
110             ctrl_if.cb.wdata    <= trans.wdata;
111             @(posedge ctrl_if.clk);                          // apply signals
112             while (ctrl_if.gnt !== 1'b1) begin
113                 @(posedge ctrl_if.clk);
114             end
115             ctrl_if.cb.req      <= 1'b0;                     // release req
116         endtask
```

Monitor

The key difference in the monitors is the use of a specialized transaction that made communication straight forward. Having multiple monitors (one for each master + one for the reset) added complexity to the scoreboard model but UVM structures made this a minor problem.

Scoreboard

In terms of implementation there is a big difference, but the main logic stayed the same. The only real difference is the ability to manage the reset signal and the duty to stop the simulation when the last transaction is processed.

4.2 Added

The UVM architecture needed some structural blocks and to get a complete testbench some new features were added. The added component will be explored in the following subsections.

Test, Environment, Agent

To complete the structure of UVM these structures were added, they are the node of the tree of modules that connect all the other components. This creates a modular code that enables easy changes if necessary, and makes adding new features fast and uncomplicated.

Coverage + Checking

To complete the testbench all the necessary coverage points have been added: from the hardware coverage to the scoreboard coverage. A checker was also added to check the hardware wires behavior.

Chapter 5

Commands

A Makefile has been used to make the compilation and execution of the testbench. Changing the parameter `TEST` it is possible to run 3 different tests as per request.

5.1 Compilation

`comp` was used to make the compilation.

```
9  comp:
10   vlib work
11   vmap work work
12   vlog -work work -sv -timescale "1ns/1ps" +incdir+. \
13     simple_mem_ctrl.sv \
14     tb_uvm_simple_mem_ctrl.sv
```

5.2 Run

`sim` was used to make the simulation and was configured using 3 parameters.

- `TEST` the name of the test to perform [ctrl_det_test, ctrl_rnd_test, ctrl_combined_test]
- `VERBOSITY` the verbosity level of the log [UVM_LOW, UVM_MEDIUM, UVM_HIGH]
- `SEED` the seed of the simulation [random, seed]

```
16 sim:
17   vsim -voptargs="+acc" -c -do "run␣-all;␣exit" \
18     -sv_seed $(SEED) \
19     +UVM_TESTNAME=$(TEST) \
20     +UVM_VERBOSITY=$(VERBOSITY) \
21     work.tb_ctrl
```

Appendix A

Appendix Chapter

A.1 Use of tools

- Visual Studio Code: systemverilog IDE.
- QuestaSim: CLI tester.
- GTKwave: wave visualizer.

A.2 Use of time

A total of 20 h distributed as follows.

- Understanding the problem: 1h.
- Planning: 3h.
- Implementation: 7h.
- Testing: 2h.
- Debugging: 3h.
- Report: 4h.

A.3 Use of AI

- Gemini (Pro): sentence enhancing, bug fixer